

LeetCode 1254: Closed Island

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 22, 2026

Original Thoughts & Strategy

"I'm using the same approach as number of enclaves.

Waha par number of each cell count krne the, yaha pure island ko hi 1 maan ke chalna he.

*Basically har border pe agar koi land mila to uske saare attatched land ko water kr denge,
kyunki such a piece of land can't be closed."*

Step-by-Step Logic (Hinglish Notes)

Step 1: Eliminate Boundary Islands (Khule Islands Hatao)

Sabse pehle grid ke charo borders (rows & cols) par traverse karo. Agar koi bhi land (0) border par milta hai, toh iska matlab hai ki woh open island ka part hai aur completely paani se nahi ghira hai. BFS/Queue ka use karke is border wale 0 aur is se jude saare connected land cells ko water (1) mein convert kar do.

Step 2: Process the Enclosed Lands (Bache hue lands ko count karo)

Step 1 execute hone ke baad, grid me jo bhi land (0) bach jayega, wo strictly paani (water) se ghira hoga (guaranteed closed island). Kyunki humne open wale already hata diye hain.

Step 3: Sink & Count (Ans badhao aur dubaa do)

Ab bas poore grid ko loop karo, jaha pehli baar 0 mile waha ans++ kardo, aur us pure closed island par BFS chala kar usko bhi 1 mark kar do taaki same island aage loop me dubara count na ho jaye. Result ans me store ho jayega!

C++ Solution Code

```

class Solution {
public:
    int closedIsland(vector<vector<int>>& grid) {

        // im using the same approach as number of enclaves
        // waha par number of each cell count krne the yaha pure island ko hi 1
        // basically har border pe agar koi land mila to uske saare attached la
        // kyunki such a piece of land cant be closed

        int n=grid.size();
        int m=grid[0].size();

        vector<pair<int,int>> dir={{-1,0},{1,0},{0,1},{0,-1}};
        queue<pair<int,int>> q;

        for(int i=0;i<n;i++){

            if(grid[i][0]==0){
                grid[i][0]=1;
                q.push({i,0});
            }

            if(grid[i][m-1]==0){
                grid[i][m-1]=1;
                q.push({i,m-1});
            }

        }

        for(int j=0;j<m;j++){

            if(grid[0][j]==0){
                grid[0][j]=1;
                q.push({0,j});
            }

            if(grid[n-1][j]==0){
                grid[n-1][j]=1;
                q.push({n-1,j});
            }

        }

        while(!q.empty()){
            int r=q.front().first;
            int c=q.front().second;
            q.pop();

```

```

        for(auto it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m){
                if(grid[nr][nc]==0){
                    grid[nr][nc]=1;
                    q.push({nr,nc});
                }
            }
        }
    }

    // now lets calculate islands left , they are guarenteed to be closed as
    int ans=0;
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            if(grid[i][j]==0){
                ans++;
                grid[i][j]=1;
                q.push({i,j});
            }

            while(!q.empty()){
                int r=q.front().first;
                int c=q.front().second;
                q.pop();

                for(auto it:dir){
                    int nr=r+it.first;
                    int nc=c+it.second;

                    if(nr>=0 && nc>=0 && nr<n && nc<m){
                        if(grid[nr][nc]==0){
                            grid[nr][nc]=1;
                            q.push({nr,nc});
                        }
                    }
                }
            }
        }
    }

    return ans;
}

```

```
}  
};
```